Building an End-to-End ML System for Iris Classification

A Comprehensive Guide to Implementing and Integrating Machine Learning Models

Project Report: End-to-End Machine Learning System for Iris Classification

1. Introduction

This project implements an end-to-end Machine Learning system for Iris classification. The integration of various components is crucial to demonstrate production-level ML engineering practices. The key components include:

- **Model development** using scikit-learn
- **API deployment** with Flask
- **Testing** through pytest
- **Code quality** ensured by Pylint
- **Containerization** via Docker
- **CI/CD** using GitHub Actions

The primary objective is to exhibit comprehensive ML engineering practices beyond mere model training.

2. Version Control Strategies

2.1 Tool Used

- **Git** for version control
- **GitHub** for repository hosting

2.2 Branching Strategy

A main-based workflow was employed:

- **main**: Production-ready code

- **Feature branches** (optional): Development phase

Workflow: feature → test → merge → main

2.3 Commit Practices

Commit practices were structured to be small and atomic with meaningful messages:

- feat: Add flask API
- fix: Resolve Docker networking issue
- test: Add API unit tests

2.4 Version Control Benefits

- **Tracks changes** in ML code
- Enables **rollback**
- Supports **team collaboration**
- Integrates with **CI/CD pipeline**

3. Testing Approach

3.1 Tool Used

- **pytest** for testing

3.2 Testing Strategy

- **Unit Testing** for model functions:
  - Data loading
  - Training
  - Evaluation
  - Prediction
- **API Testing** for endpoint validation:
  - /predict
  - /health check

3.3 Key Techniques

- **Fixtures**: Reusable test data
- **Assertions**: Validate outputs
- **Isolation**: Test each function independently

3.4 Example

```
def test_evaluate_model_accuracy():
  assert accuracy > 0.8
```

3.5 Benefits

- **Prevents regression**
- **Ensures correctness**
- Enables **CI automation**

4. Code Quality (Linting)

Tool Used

- **Pylint**

Purpose

- Enforce **coding standards**
- Detect **errors early**
- Improve **readability**

Issues Handled

- Import errors (src layout)
- ML library false positives
- Exception handling improvements
- Formatting issues

Outcome

- **Code quality score** improved to ~10/10

5. CI/CD Pipeline

5.1 Tool Used

- **GitHub Actions**

5.2 CI Pipeline

Triggered on push:

1. Install dependencies
2. Train model
3. Run tests (pytest)
4. Run linting (Pylint)

5.3 CD Pipeline

- Build Docker image
- Tag image (latest + commit SHA)
- Push to Docker Hub

5.4 Workflow Diagram

Code Push → CI (Test + Lint) → Build Docker → Push Image

6. Containerization

Tool Used

- **Docker**

Purpose

- Ensure **environment consistency**
- Simplify **deployment**
- Improve **scalability**

Key Features

- Lightweight Python image
- Pre-trained model inside container
- Gunicorn for production serving

Deployment Flow

Docker → Container → Flask API → ML Model

7. API Deployment

Tool Used

- **Flask**

Features

- /predict endpoint
- JSON input/output
- Error handling

Example Request

```
{
  "features": [5.1, 3.5, 1.4, 0.2]
}
```

8. Best Practices Applied

Engineering Practices

- Modular code structure (src/)
- Separation of concerns

- Environment isolation

Testing Practices

- Automated testing
- Test coverage for core logic
- API validation

CI/CD Practices

- Automated builds
- Quality gates (lint + test)
- Secure Docker authentication (token)

Deployment Practices

- Containerized application
- Production server (Gunicorn)
- Port exposure and networking

9. Challenges Faced

Challenge	Solution
Import errors (src/)	Added PYTHONPATH
Docker networking	Used 0.0.0.0
Pylint ML issues	Disabled no-member
API crash (missing model)	Pre-trained model

10. Conclusion

This project successfully demonstrates a complete ML engineering pipeline, encompassing:

- Model development
- Testing
- Code quality
- CI/CD automation
- Containerized deployment

It reflects industry best practices for building scalable and maintainable ML systems.